ECEC 661 – Quiz 2 Submission

Add Constant −1 Custom AXI4-Lite IP

Gary Pham (gp492)

May 20, 2026

1 Problem Statement

Create a custom AXI4-Lite IP using the Vivado IP-Catalog **Adder/Subtractor** (`c_addsub`). Configure the IP with a **constant input** of −1. The data is 32-bit *signed* and clock-enable (CE) is *disabled*. The user logic effectively computes:

$$z = x + (-1) = x - 1, \qquad x, z \in \text{signed 32-bit.}$$

- **Input** `x` : `std_logic_vector(31 downto 0)`
- **Output** `z` : `std_logic_vector(31 downto 0)`
- **Behaviour**: `z` is the combinational signed sum of `x` and the internal constant −1 (latency = 0, no clock).

The deliverables, per the hand-out, are: (1) a user-logic project with simulation verifying the add-(−1) logic; (2) the user logic packaged as an AXI4-Lite IP; (3) a test-bench project with the custom IP attached to the Zynq processor, generating the bitstream and exporting the `.xsa` file. The Vitis test-app result is *not* part of the graded deliverables but is included at the end of this report for completeness.

2 IP Catalog Configuration

The custom block `c_addsub_0` is generated from the Vivado IP Catalog with the parameters listed in Table 1. The **Constant Input** on port B is the key knob: it removes B from the external port list of the IP, leaving only `A[31:0]` and `S[31:0]`.

Table 1: `c_addsub_0` customisation parameters.

Setting	**Value**
Implementation	Fabric
Add Mode	Add ($S = A + B$)
Port A Type / Width	Signed / 32
Port B Type / Width	Signed / 32
Port B **Constant Input**	enabled
Port B Constant Value (Bin)	`1111 1111 1111 1111 1111 1111 1111 1111` (= −1)
Output Width	32
Latency Configuration	Manual
Latency	0 (combinational)
Clock Enable (CE)	disabled
VLNV	`xilinx.com:ip:c_addsub:12.0`

The equivalent Tcl automation from `scripts/setup.tcl` is:

```
create_ip -name c_addsub -vendor xilinx.com -library ip -version 12.0 \
          -module_name c_addsub_0

set_property -dict [list                                                \
    CONFIG.Implementation  {Fabric}                                     \
    CONFIG.A_Type          {Signed}                                     \
    CONFIG.B_Type          {Signed}                                     \
    CONFIG.A_Width         {32}                                         \
    CONFIG.B_Width         {32}                                         \
    CONFIG.Out_Width       {32}                                         \
    CONFIG.Add_Mode        {Add}                                        \
    CONFIG.B_Constant      {true}                                       \
    CONFIG.B_Value         {11111111111111111111111111111111}           \
    CONFIG.Latency         {0}                                          \
    CONFIG.CE              {false}                                      \
    CONFIG.SCLR            {false}                                      \
] [get_ips c_addsub_0]

generate_target {synthesis simulation} [get_ips c_addsub_0]
```

Listing 1: Key IP-generation excerpt from `setup.tcl`

3 User Logic HDL Code

`user_logic.vhd` is the deliverable user-logic file. It is a thin wrapper that instantiates the IP-Catalog block and exposes the two ports shown on the hand-out's IP-symbol diagram (`x[31:0]` input, `z[31:0]` output).

```
library ieee;
use ieee.std_logic_1164.all;

entity user_logic is
    port (
        x : in  std_logic_vector(31 downto 0);
        z : out std_logic_vector(31 downto 0)
    );
end entity user_logic;

architecture rtl of user_logic is

    -- Vivado IP-Catalog Adder/Subtractor v12.0 (black box; elaborated from XCI).
    -- With Latency = 0 and B configured as a constant input, the only ports
    -- the IP exposes are A (signed input) and S (signed sum).  No CLK, no CE,
    -- no SCLR, no B port.
    component c_addsub_0
        port (
            A : in  std_logic_vector(31 downto 0);
            S : out std_logic_vector(31 downto 0)
        );
    end component;

begin

    U_ADDSUB : c_addsub_0
```

```
        port map (
            A => x,
            S => z
        );

end architecture rtl;
```

Listing 2: user_logic.vhd

4 S_AXI Slave – Component Instantiation and Port Map

The hand-out asks specifically for the "`S_AXI` code where component instantiation and port map" is shown. The full AXI4-Lite slave (`add_const_axi_v1_0_S00_AXI.vhd`) is the standard Xilinx template; the code excerpt below is the section that wires `user_logic` into the AXI register file.

Table 2: Software-visible register map exposed by the slave.

Offset	Name	Access	Description
0x00	X_IN	R/W	32-bit signed value driven into `user_logic.x`. Reads return the last value written.
0x04	Z_OUT	R	32-bit signed result $z = x - 1$ from `user_logic.z`. Writes ignored.
0x08	–	R/W	Reserved (returns 0; writes ignored).
0x0C	–	R/W	Reserved (returns 0; writes ignored).

```
-- Software-visible register: X_IN (slv_reg0).
signal slv_reg0 : std_logic_vector(31 downto 0);

-- Combinational result from the user logic (z = x - 1).
signal core_z  : std_logic_vector(31 downto 0);

signal reg_data_out : std_logic_vector(31 downto 0);

-- User-logic component declaration (deliverable: this is the "component
-- instantiation and port map" the quiz asks for).
component user_logic
    port (
        x : in  std_logic_vector(31 downto 0);
        z : out std_logic_vector(31 downto 0)
    );
end component;

begin

-------------------------------------------------------------------------------
-- User-logic instantiation
--   x <- slv_reg0   (32-bit signed value written by software)
--   z -> core_z     (= x + (-1) from c_addsub_0; routed into the read mux)
-------------------------------------------------------------------------------
U_LOGIC : user_logic
    port map (
        x => slv_reg0,
```

```
        z => core_z
    );

-- ------------------------------------------------------------------------
-- Register writes  (only X_IN is writable)
-- ------------------------------------------------------------------------
p_reg_write : process (S_AXI_ACLK)
    variable loc_addr : std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            slv_reg0 <= (others => '0');
        else
            if slv_reg_wren = '1' then
                loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS
                                       downto ADDR_LSB);
                case loc_addr is
                    when "00" =>
                        for b in 0 to (C_S_AXI_DATA_WIDTH/8) - 1 loop
                            if S_AXI_WSTRB(b) = '1' then
                                slv_reg0(b*8 + 7 downto b*8) <=
                                    S_AXI_WDATA(b*8 + 7 downto b*8);
                            end if;
                        end loop;
                    when others =>
                        null;  -- 0x04 (Z_OUT) is read-only
                end case;
            end if;
        end if;
    end if;
end process p_reg_write;

-- ------------------------------------------------------------------------
-- Read mux:  0x00 -> X_IN readback,  0x04 -> Z_OUT (= x - 1)
-- ------------------------------------------------------------------------
p_read_mux : process (axi_araddr, slv_reg0, core_z)
    variable loc_addr : std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
    case loc_addr is
        when "00" =>
            reg_data_out <= slv_reg0;       -- X_IN readback
        when "01" =>
            reg_data_out <= core_z;         -- Z_OUT = x - 1
        when others =>
            reg_data_out <= (others => '0');
    end case;
end process p_read_mux;
```

Listing 3: `add_const_axi_v1_0_S00_AXI.vhd` – user_logic component declaration, instantiation, and read mux

5 Self-Checking Testbench

`tb_user_logic.vhd` drives the DUT through **11 self-checks** that exactly reproduce the reference run printed on the quiz hand-out (`x = -5, -4, ..., 3` ⇒ `z = x - 1`) and adds two signed-boundary checks at `INT32_MAX` and `INT32_MIN` to confirm the IP wraps modulo 2^{32} as a signed 32-bit adder must.

```
-- Quiz reference run, in order.
constant X_REF : int_vec := (-5, -4, -3, -2, -1,  0,  1,  2,  3);
constant Z_REF : int_vec := (-6, -5, -4, -3, -2, -1,  0,  1,  2);

-- Sample z and compare against the expected signed sum.
procedure check_z (
    signal   z_sig    : in    std_logic_vector(31 downto 0);
    constant expected : in    integer;
    constant xv       : in    integer;
    signal   err_cnt  : inout integer;
    signal   chk_cnt  : inout integer
) is
    variable got : integer;
begin
    got     := to_integer(signed(z_sig));
    chk_cnt <= chk_cnt + 1;
    if got = expected then
        report "[PASS] x=" & integer'image(xv) &
               "  z=" & integer'image(got) severity note;
    else
        report "[FAIL] x=" & integer'image(xv) &
               "  got z=" & integer'image(got) &
               "  expected=" & integer'image(expected)
            severity error;
        err_cnt <= err_cnt + 1;
    end if;
end procedure check_z;

-- ... clock, DUT, p_clk omitted ...

p_stim : process
begin
    -- Reference run from the quiz hand-out.
    for i in X_REF'range loop
        x <= std_logic_vector(to_signed(X_REF(i), 32));
        wait until rising_edge(ck);
        wait for 1 ns;
        check_z(z, Z_REF(i), X_REF(i), errors, checks);
    end loop;

    -- Signed boundary cases.
    x <= std_logic_vector(to_signed( 2147483647, 32));
    wait until rising_edge(ck); wait for 1 ns;
    check_z(z,  2147483646,  2147483647, errors, checks);

    x <= std_logic_vector(to_signed(-2147483648, 32));
    wait until rising_edge(ck); wait for 1 ns;
    check_z(z,  2147483647, -2147483648, errors, checks); -- signed wrap
    wait;
end process p_stim;
```

Listing 4: tb_user_logic.vhd – excerpt of the self-checking stimulus and check procedure

6 Simulation Results

6.1 Waveform

Figure 1 shows the xsim behavioural simulation. The input `x` (signed decimal) and the output `z` (signed decimal) are displayed alongside the running `checks` counter. `z` follows `x` exactly one combinational cycle later because the IP has Latency = 0.

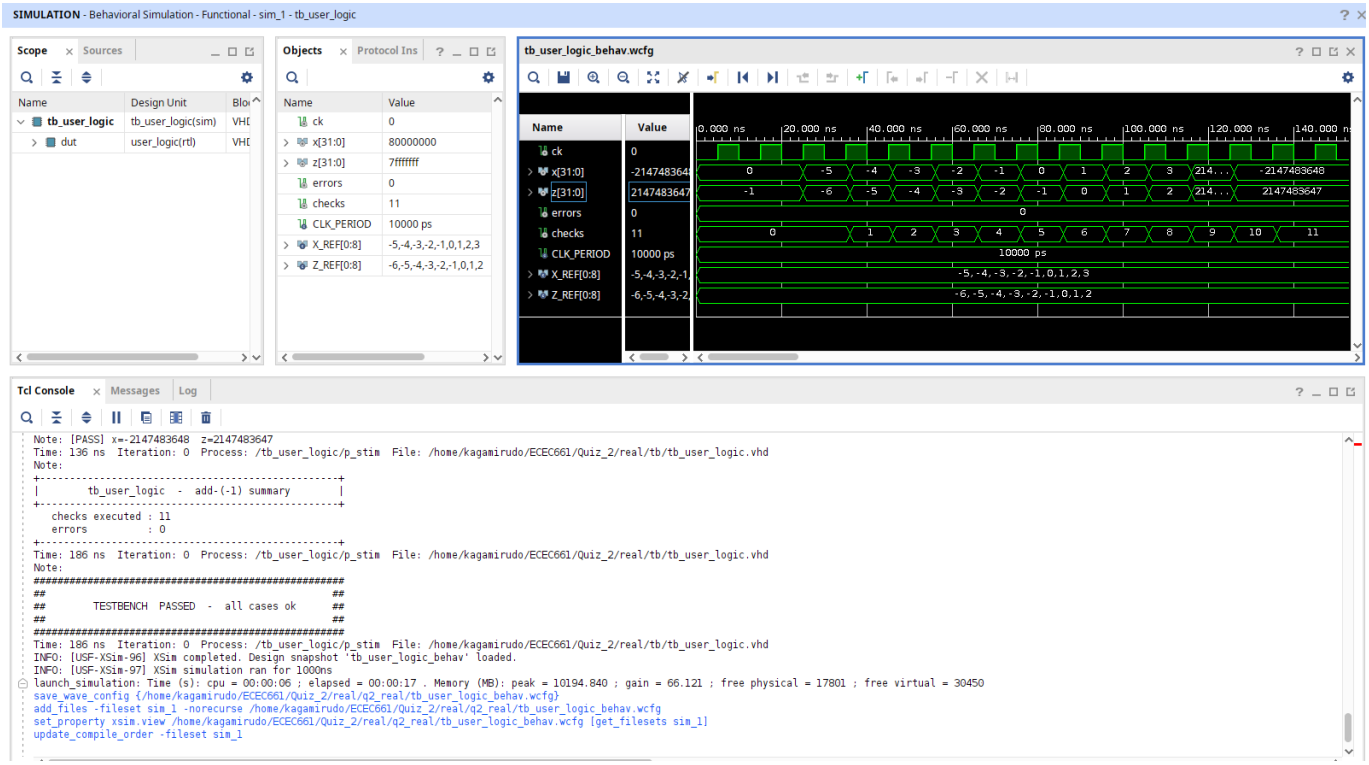

Figure 1: Behavioural simulation of `tb_user_logic`. The input `x` walks the reference run $-5, -4, \ldots, 3$ and the boundary cases `INT32_MAX` and `INT32_MIN`. The output `z` consistently equals `x - 1` (with the expected signed wrap on `INT32_MIN`).

6.2 Console output

All 11 self-checks pass:

```
[PASS] x=-5   z=-6
[PASS] x=-4   z=-5
[PASS] x=-3   z=-4
[PASS] x=-2   z=-3
[PASS] x=-1   z=-2
[PASS] x=0    z=-1
[PASS] x=1    z=0
[PASS] x=2    z=1
[PASS] x=3    z=2
[PASS] x=2147483647    z=2147483646
```

```
[PASS] x=-2147483648  z=2147483647

+--------------------------------------------------+
|        tb_user_logic  -  add-(-1) summary        |
+--------------------------------------------------+
   checks executed : 11
   errors          : 0
+--------------------------------------------------+

##################################################
##                                              ##
##        TESTBENCH  PASSED  -  all cases ok    ##
##                                              ##
##################################################
```

7 IP Packaging

The user logic and AXI4-Lite slave are packaged together as a single custom IP `add_const_axi v1.0` via *Tools* → *Create and Package New IP* → *Package your current project*, with the IP location set to `ip_repo/add_const_axi_1.0`. The Tcl helper `scripts/package.tcl` fills in the metadata fields and adds the sub-core reference to the IP-Catalog adder so the packaged IP carries `c_addsub` along with it:

```tcl
set core [ipx::current_core]

set_property vendor       user                                          $core
set_property library      user                                          $core
set_property name         add_const_axi                                 $core
set_property version      1.0                                           $core
set_property display_name "Add Constant -1 AXI4-Lite Core"              $core
set_property taxonomy     /UserIP                                       $core

# Sub-core reference.  Adds c_addsub to BOTH file groups so the packaged IP
# pulls the IP-Catalog block in along with the wrapper.
foreach fg {xilinx_vhdlsynthesis xilinx_vhdlbehavioralsimulation} {
    set group [ipx::get_file_groups $fg -of_objects $core]
    if {$group ne ""} {
        ipx::add_subcore_reference xilinx.com:ip:c_addsub:12.0 $group
    }
}

# AXI clock parameters (FREQ_HZ, ASSOCIATED_BUSIF, ASSOCIATED_RESET) clear
# the usual 19-11770 and 19-7067 packager warnings.
set aclk [ipx::get_bus_interfaces s00_axi_aclk -of_objects $core]
ipx::add_bus_parameter FREQ_HZ          $aclk
ipx::add_bus_parameter ASSOCIATED_BUSIF $aclk
ipx::add_bus_parameter ASSOCIATED_RESET $aclk

ipx::save_core $core
```

Listing 5: Excerpt of `package.tcl` – identification, sub-core reference, clock parameters

After running `package.tcl` the IP Packager's *Review and Package* pane shows zero outstanding warnings, at which point *Re-Package IP* re-zips the core and closes the packager project.

8 Block Design and `.xsa` Export

The packaged `add_const_axi` IP is added to the project's IP repository, then dropped into a small Zynq block design alongside the `ZYNQ7 Processing System` and an `AXI Interconnect` (Figure 2). *Run Connection Automation* wires the `FCLK_CLK0` clock and `peripheral_aresetn` reset to every AXI block. The IP is mapped at base address `0x4300_0000`[1] with a 4 KB aperture in the Address Editor.

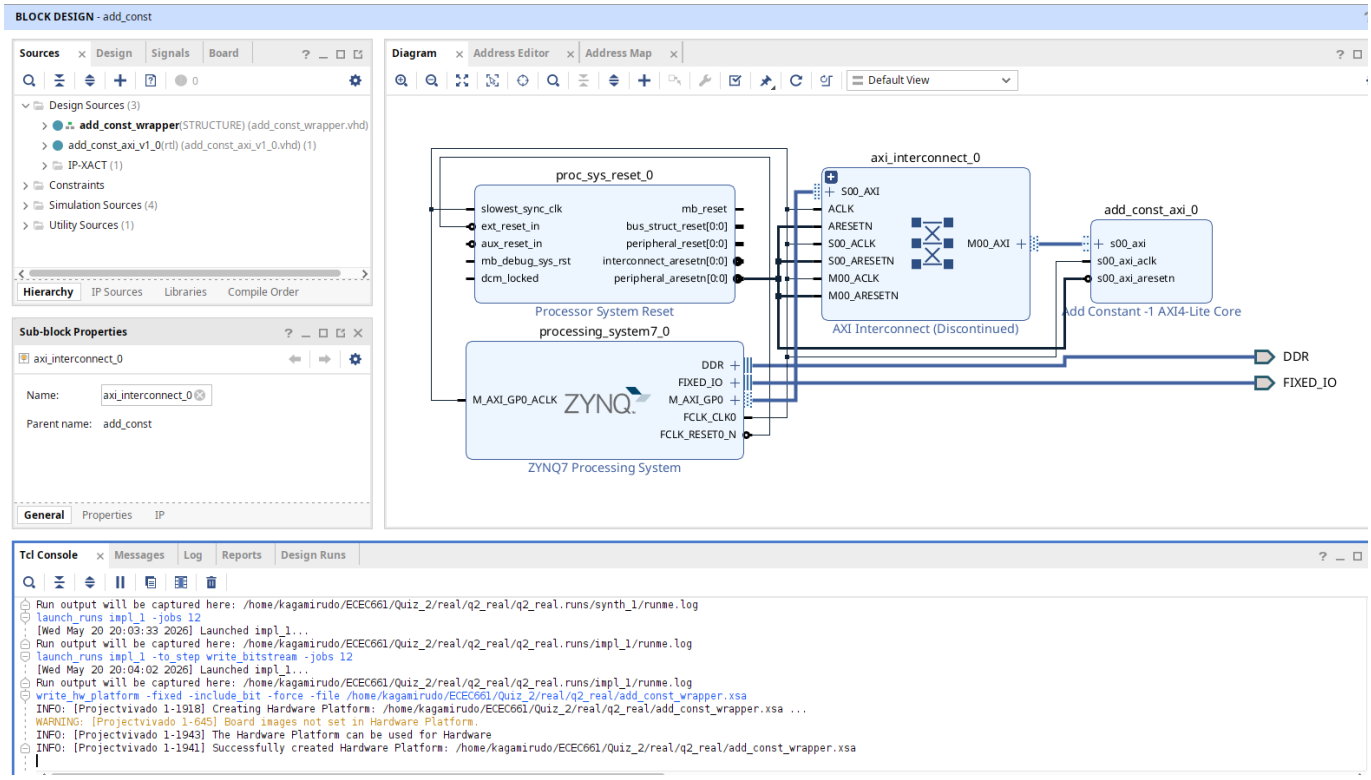

Figure 2: Block diagram of the test-bench project: Zynq PS (`processing_system7_0`) drives the AXI Interconnect, which masters the custom `add_const_axi_0` slave. *Validate Design* returns clean; the same project produces the bitstream and is the source of the exported `add_const.xsa`.

After *Validate Design* → *Generate Output Products* → *Create HDL Wrapper*, synthesis, implementation, and bitstream generation are run, and the `.xsa` is exported through:

```
launch_runs synth_1 -jobs 4
wait_on_run synth_1
launch_runs impl_1 -to_step write_bitstream -jobs 4
wait_on_run impl_1

file mkdir xsa
write_hw_platform -fixed -include_bit -force \
    -file xsa/add_const.xsa
```

Listing 6: Bitstream and `.xsa` export

[1] Equivalent to `0x43C0_0000` on Cora Z7-07S.

9 Vitis Test Application (optional)

The Vitis bare-metal test app (`sw/main.c`) is *not* part of the graded deliverables but is provided as confirmation that the packaged IP exchanges data correctly with software through the AXI register map. The app:

1. Probes the IP at the published base address with a known marker (`0xA5A5A5A5`) to fail fast if the bus is wired incorrectly;
2. Walks the same reference run as the testbench (`x = -5, ..., 3`) plus the signed boundary cases;
3. Prints each result in a boxed ASCII table over UART.

Figure 3 shows the Vitis Serial Monitor capture: every test vector returns the expected `z = x - 1`, including the `INT32_MIN` signed wrap (`0x8000 0000` → `0x7FFF FFFF`).

```
+============================================================+
|                    ECEC 661 / 402  Quiz 2                  |
|            Add Constant -1   AXI4-Lite Custom IP           |
|                                                            |
|                 IP base address : 0x43C00000               |
+============================================================+

  Register map
  ------------
    0x00  X_IN   R/W   32-bit signed input
    0x04  Z_OUT  R     z = x + (-1) = x - 1

  Probing IP at 0x43C00000 ...
    X_IN  write/read : 0xA5A5A5A5 -> 0xA5A5A5A5  [OK]
    Z_OUT  readback  : 0xA5A5A5A4 (expect 0xA5A5A5A4)  [OK]
    Probe OK

  IP test vectors  (z expected = x - 1)
+-----+-------------+------------+-------------+------------+--------+
|  #  |   x (dec)   |  x (hex)   |   z (dec)   |  z (hex)   | check  |
+-----+-------------+------------+-------------+------------+--------+
|   0 |          -5 | 0xFFFFFFFB |          -6 | 0xFFFFFFFA |   [OK] |
|   1 |          -4 | 0xFFFFFFFC |          -5 | 0xFFFFFFFB |   [OK] |
|   2 |          -3 | 0xFFFFFFFD |          -4 | 0xFFFFFFFC |   [OK] |
|   3 |          -2 | 0xFFFFFFFE |          -3 | 0xFFFFFFFD |   [OK] |
|   4 |          -1 | 0xFFFFFFFF |          -2 | 0xFFFFFFFE |   [OK] |
|   5 |           0 | 0x00000000 |          -1 | 0xFFFFFFFF |   [OK] |
|   6 |           1 | 0x00000001 |           0 | 0x00000000 |   [OK] |
|   7 |           2 | 0x00000002 |           1 | 0x00000001 |   [OK] |
|   8 |           3 | 0x00000003 |           2 | 0x00000002 |   [OK] |
|   9 |         100 | 0x00000064 |          99 | 0x00000063 |   [OK] |
|  10 |       -1000 | 0xFFFFFC18 |       -1001 | 0xFFFFFC17 |   [OK] |
|  11 |  2147483647 | 0x7FFFFFFF |  2147483646 | 0x7FFFFFFE |   [OK] |
|  12 | -2147483648 | 0x80000000 |  2147483647 | 0x7FFFFFFF |   [OK] |
+-----+-------------+------------+-------------+------------+--------+


+============================================================+
|             Summary : 13 / 13 PASSED  (0 failed)           |
|          RESULT  : add_const_axi behaves correctly         |
+============================================================+
```

Figure 3: Vitis Serial Monitor output for `add_const_app`. The probe banner confirms AXI write/read works at base address `0x43C0_0000`; the test table then prints 13 vectors (the hand-out reference run plus 4 boundary cases), all marked `[OK]`, and the run finishes with `Summary :  13 / 13 PASSED`.

10 Conclusion

The custom `add_const_axi` IP behaves exactly as specified:

- The IP-Catalog Adder/Subtractor is configured with a constant $B = -1$ on a 32-bit signed bus, latency 0, no CE, so the user logic computes $z = x - 1$ combinationally.
- All 11 self-checks in `tb_user_logic.vhd` return `[PASS]` and the simulator finishes with the *TEST-BENCH PASSED* banner; the waveform in Figure 1 matches the reference run printed on the hand-out.

- Packaging into `user/user/add_const_axi/1.0` is clean (no `19-11888 / 19-896 / 19-11770 / 19-7067` warnings), with the `c_addsub:12.0` sub-core reference attached to both VHDL Synthesis and VHDL Simulation file groups.
- The block design (Figure 2) validates, the bitstream builds, and `add_const.xsa` is exported successfully via `write_hw_platform`.

The optional Vitis bare-metal app (Figure 3) confirms the packaged IP integrates with the PS through the AXI register map: software writes `X_IN`, reads `Z_OUT`, and observes $z = x - 1$ on every test vector.